

字符串专题

lmh_qwq

2026.10.6

字符串

我们采用如下的记号：

大写字母（例如 S, T ）表示字符串，小写字母表示字符；字符串 S 的长度为 $|S|$ 。默认 $n = |S|, m = |T|$ 。

S_i 在 $1 \leq i \leq |S|$ 的情况下表示 S 的第 i 个字符；否则，默认 $S_i = \epsilon$ 表示空字符。

令 Σ 表示字符集。如果没有特殊说明，本文中的字符串均由拉丁字母组成，并且小写字母和大写字母不会同时出现。此时字符集大小为常数。

定义两个字符串 S, T 相等当且仅当 $|S| = |T|$ 并且对于所有的 i 都有 $S_i = T_i$ ；定义 S 的字典序小于 T 的字典序当且仅当存在一个 i 满足 $S_i < T_i$ 并且 $\forall j < i, S_j = T_j$ 。

其中，字符的大小关系定义为： ϵ 小于其他所有字符；拉丁字母按照字典序排列；数字按照大小关系排列；分隔符大于（或者小于）所有非空字符。

令 $S + T$ 或 ST 表示将两个字符串（或字符）连接在一起得到的字符串。例如 $S = S_1 S_2 \cdots S_{|S|}$ 。

我们用 $S_{l,r} (1 \leq l \leq r \leq |S|)$ 代表 $S_l S_{l+1} \cdots S_r$ ，也即 S 中第 $l \sim r$ 个字符顺次连接得到的**子串**。

定义 S 的**前缀**为所有 $S_{1,i}$ 组成的集合，**后缀**为 $S_{i,|S|}$ 组成的集合，其中 $1 \leq i \leq |S|$ 。
真前缀（真后缀）在前后缀的基础上要求不能等于原字符串。

字符串匹配

这是最经典的一个字符串问题：给定字符串 S, T ，找出所有 T 在 S 中出现的位置。

朴素的算法是：枚举 T 所有可能的位置，暴力枚举判断是否匹配。时间复杂度为 $O(nm)$ ，效率很低。

哈希

考虑哈希算法。哈希算法的思想是：对于难以直接比较两两之间是否相同的一类元素，可以将它们映射到容易比较的一类元素（一般是整型变量），再进行比较。

当然这么做一般不是确定性的，因为不同的元素可能被映射到相同的值（哈希冲突）。所以一个高效的哈希映射应该让哈希冲突的概率尽可能低。

对于字符串，设计哈希算法是比较简单的。比如我们可以 $\mathbf{a} \rightarrow 1, \mathbf{b} \rightarrow 2, \dots, \mathbf{z} \rightarrow 26$ 然后把整个字符串看作 27 进制数；为了防止数值过大，可以把这个数模 $10^9 + 7$ 。

一般的方法是：首先选择大小适中的 B （略大于 $|\Sigma|$ ，和 P 互质）和大质数 P （为了方便计算，有时也会取 $P = 2^{64}$ ）。接下来任取 $\Sigma \rightarrow \{1, \dots, B - 1\}$ 的单射 f （可以直接用强制类型转换），令

$$\text{hash}(S) = \left(\sum_{i=1}^n f(S_i) B^{n-i} \right) \bmod P。$$

对于两个随机字符串，哈希冲突概率约为 $1/P$ 。

根据

$$\text{hash}(S) = \left(\sum_{i=1}^n f(S_i) B^{n-i} \right) \bmod P$$

可以简单地增量计算字符串的哈希值：

$$\text{hash}(S + c) = (B \cdot \text{hash}(S) + f(c)) \bmod P;$$

$$\text{hash}(c + S) = (B^{|S|} \cdot f(c) + \text{hash}(S)) \bmod P。$$

所以对于串 S 可以递推出所有前缀 $S_{1,i}$ 的哈希值。然后可以用

$$\text{hash}(S_{l,r}) = (\text{hash}(S_{1,r}) - B^{r-l+1} \cdot \text{hash}(S_{1,l-1})) \bmod P$$

计算所有子串的哈希值。

生日悖论

如果我们比较两个字符串，哈希冲突的概率约为 $1/P$ ；但是如果我们要统计字符串种类数，那么任意两个字符串之间如果哈希冲突都会导致结果算错。 k 个字符串有 $O(k^2)$ 个 pair，直觉上出错概率会大大增加。

事实上这是经典的生日悖论模型，可以证明出错的概率约为

$$1 - \exp\left(-\frac{k(k-1)}{2P}\right)$$

如果 P 在 10^9 级别，那么很容易挂掉。

解决方法是使用自然溢出或者双哈希，让单次错误概率在 10^{-18} 级别，这样正常的数据范围就卡不掉了。

例题：P5546 [POI 2000 R3] 公共串

给定 n 个字符串 $S_i (1 \leq i \leq n)$ ，求出它们的最长公共子串。

$n \leq 5, |S_i| \leq 2000$ 。

例题：P5546 [POI 2000 R3] 公共串

给定 n 个字符串 $S_i (1 \leq i \leq n)$ ，求出它们的最长公共子串。

$n \leq 5, |S_i| \leq 2000$ 。

哈希算法模板题（当然做法远不止这一种）。

可以二分这个长度 k ，算出每个 S_i 所有长为 k 的子串的哈希值。然后对这些集合求交即可。可以开一个 `unordered_map`，如果某个哈希值在 S_i 中出现过就按位或 2^i 。

令 $L = \sum |S_i|$ ，时间复杂度 $O(L \log L)$ 。

例题：P1117 [NOI2016] 优秀的拆分

如果一个字符串可以被拆分为 $AABB$ 的形式，其中 A 和 B 是任意**非空**字符串，则我们称该字符串的这种拆分是优秀的。

例如，对于字符串 `aabaabaa`，如果令 $A = \text{aab}$ ， $B = \text{a}$ ，我们就找到了这个字符串拆分成 $AABB$ 的一种方式。

一个字符串可能没有优秀的拆分，也可能存在不止一种优秀的拆分。

比如我们令 $A = \text{a}$ ， $B = \text{baa}$ ，也可以用 $AABB$ 表示出上述字符串；但是，字符串 `abaabaa` 就没有优秀的拆分。

现在给出一个长度为 n 的字符串 S ，我们需要求出，在它所有子串的所有拆分方式中，优秀拆分的总个数。

每个测试点至多 10 组数据， $n \leq 30000$ 。

做这样一个转化。令 f_i 表示 $S_{1,i}$ 中形如 AA 的后缀个数， g_i 表示 $S_{i,n}$ 中形如 AA 的前缀个数。则最终答案为

$$\sum_{i=1}^n f_i g_{i+1}.$$

现在需要找出 S 中所有的循环段。

枚举循环节的长度 d 。暴力做需要比较 $n - 2d + 1$ 次，如何做到更好呢？

我们发现：如果一个字符串周期为 d ，说明它往右平移 d 个位置之后能和自己匹配。

将所有 d 的倍数设为关键点，一个至少循环两次的片段一定包含至少两个关键点；想象在一个关键点处把这个片段拿起来拖到旁边一个关键点的位置，平移过后对应位置和原来的片段匹配。

可以从相邻两个关键点 x, y 开始求 $S_{x,n}, S_{y,n}$ 的最长公共前缀以及 $S_{1,x-1}, S_{1,y-1}$ 最长公共后缀。如果二者长度之和 $\geq d$ 说明找到了一个循环片段；同时也找到了这个片段的起点和终点。

假设这个片段是 $[L, R]$ ，那么 $f_{L+2d-1} \sim f_R$ 全部需要 $+1$ ，用前缀和优化。

总共需要 $\sum_i \frac{n}{i} = O(n \log n)$ 次求 LCP 操作。

注意这个算法的结论：至少重复两次的循环子串数量（以及循环次数之和）不会太多，而且可以简单地求出。

如何快速求出 LCP 呢？

可以用 SA 做到 $O(1)$ ，但更简单的办法是二分哈希。也就是：二分这个公共前缀的长度，用哈希判断是否匹配。

所以单次操作需要 $O(\log n)$ ，总时间复杂度 $O(n \log^2 n)$ 。

Trie

在处理多个字符串的时候，有几种常用的办法。一种朴素的想法是把多个字符串拼在一起，中间用**分隔符**（有时可以全用 `#` 或者类似的特殊字符，有时需要分隔符互不相同）隔开。

另一种方法是 Trie。它的想法是：我们经常会增量地处理字符串，也就是逐步在字符串尾部添加字符。现在 Trie 就把中间每个状态存储为一个节点，“尾部加字符”的操作变成连边。例如代表 `abc` 的节点连一条编号为 `d` 的边，指向 `abcd` 节点。

这样的好处是：如果我们有一个未知字符串，要判断它是否和某个已知字符串相同，可以在 Trie 上从根节点（空串）出发沿着边走。

对于整数，可以转化为二进制之后建 01 Trie。

建树操作转化为每次添加一个字符串 S 。添加字符串时，依次处理每一个字符 $S_i (i = 1, 2, \dots, n)$ ：

- 从根节点 0（空串）出发，设当前节点为 u 。
- 每次考虑当前节点 u 和处理的字符 S_i 。若对应出边 $u \xrightarrow{S_i} v$ 不存在，则新建一个节点 v ，连边；然后走过去 ($u \leftarrow v$)。
- 将最后一步到达的 u 标记为终止节点。

一般会存储 $to_{S,c}$ 代表 $S \xrightarrow{c} to_{S,c}$ 。此时建树的时间复杂度为 $O(\sum |S_i|)$ ，空间复杂度要额外乘上字符集大小 $|\Sigma|$ 。

自动机

Trie 就是一个典型的“自动机”。一个自动机是一个有向图，存在根节点 r ，边用字符进行编号。一个节点的出边的编号不能重复。

现在有一个字符串 S 。在自动机上处理这个字符串的方法是：

- 令 $u \leftarrow r$;
- 对于 $S_1 \sim S_n$ ，依次令 $u \leftarrow to_{u,S_i}$ （其中 to_{u,S_i} 为 $u \xrightarrow{S_i}$ 指向的节点）;
- 返回最终到达的节点。

如果任意一步找不到对应的出边，需要进行错误处理（事实上一般的自动机定义不允许缺少出边，但可以新建一个错误节点把不存在的边都连过去，这样就符合定义了）。

例题：P8306 【模板】字典树 / Trie

给定 n 个模式串 $s_1, s_2, \dots, s_n$ 和 q 次询问，每次询问给定一个文本串 t_i ，请回答 $s_1 \sim s_n$ 中有多少个字符串 s_j 满足 t_i 是 s_j 的前缀。

一个字符串 t 是 s 的前缀当且仅当从 s 的末尾删去若干个（可以为 0 个）连续的字符后与 t 相同。

$$T, n, q \leq 10^5, \sum |s_i| + \sum |t_i| \leq 3 \times 10^6。$$

考虑对 $\{s_i\}$ 建 Trie 树。

在 Trie 树上，一个节点（对应的串）是另一个节点（对应的串）的前缀**当且仅当**前者是后者的祖先。

所以如果 t_i 在 Trie 树上有对应节点，那么它的子树中所有终止节点就包含它作为前缀。

预处理一下子树中有多少个终止节点（类似子树大小的算法）即可。

例题：P4551 最长异或路径

给定一棵 n 个点的带权树，结点下标从 1 开始到 n 。求树中所有异或路径的最大值。

异或路径指树上两个结点之间唯一路径上的所有边权的异或值。

$1 \leq n \leq 10^5; 0 < u, v \leq n; 0 \leq w < 2^{31}$ 。

处理树上的路径问题，常见思路是：拆成 $u \rightarrow \text{LCA}(u, v) \rightarrow v$ 两段，这样每一段都是祖先到后代。

这个问题拆完之后该如何转化？

如果你要求区间异或和，可以用前缀异或和。在这里能否使用类似的方法？

没错，推一下式子会发现异或路径的权值就是 $s_u \oplus s_v$ ，其中 s_u 表示根节点到 u 的路径异或和。

所以转化为：给定 n 个数，求两两之间异或得到的最大值。

考虑枚举第一个数，找出最优的第二个数。常见的思路是**按位贪心**：优先让高位为 1，因为如果高位赚了，那么低位不管怎么亏都亏不了更多。

假设第一个数是 0，那我们肯定希望最高位是 1，然后尝试让次高位取 1，以此类推.....也就是说，我们**增量构造答案的二进制表示的前缀**。

所以可以建 01 Trie。如果第一个数当前位为 0 就尽可能走 1；否则尽可能走 0。如果不行的话，再走另一个方向。这样就可以在 $O(\log V)$ 的时间内完成单次查询。

因而建树和查询的时间复杂度都是 $O(n \log V)$ ，总时间复杂度亦然。

例题：P3065 [USACO12DEC] First! G

Bessie 一直在研究字符串。她发现，通过改变字母表的顺序，她可以按改变后的字母表来排列字符串（字典序大小排列）。

例如，Bessie 发现，对于字符串 `omm`, `moo`, `mom` 和 `omnmom`，她可以使用标准字母表使 `mom` 排在第一个（即字典序最小），她也可以使用字母表 `abcdefghijklonmpqrstuvwxyz` 使得 `omm` 排在第一个。然而，Bessie 想不出任何方法（改变字母表顺序）使得 `moo` 或 `omnmom` 排在第一个。

请计算：通过重新排列字母表的顺序，输入中有哪些字符串可以排在第一个（即字典序最小）。

$$n \leq 30000, \sum |S_i| \leq 300000.$$

首先把 Trie 建出来，并考虑一个特定的字符串 S_i 能否成为最小的。

拉出 S_i 对应的那条链。如果存在 S_i 的真前缀 S_j 则显然无解；否则，对于链上每一个节点，它的出边当中，位于链上的那一条必须最小。

所以如果这里有 c, d 两个字符对应的出边，并且 c 在当前链中，那么连边 $c \rightarrow d$ 代表 $c < d$ 。最后如果连出 DAG 则有解，如果有环那么无解。

时间复杂度 $O(n|\Sigma|^2 + |\Sigma| \cdot \sum |S_i|)$ 。

KMP

回到之前的字符串匹配问题。哈希算法时间复杂度很优，但终究不是确定性的。有没有线性时间复杂度并且确定性的做法呢？

KMP 就是这样的一种算法。它可以在 $O(n + m)$ 的时间复杂度之内求出所有 $S_{1,i}$ 和 T 的最大匹配段。

其中 S 和 T 的最大匹配段（之后记作 $S \bmod T$ ）的定义是最长的 S 的后缀，使得它同时是 T 的前缀。下文中如果说“ S 的最大匹配段”，则默认 T 已经给定。

例如 $\text{ababa} \bmod \text{aba} = \text{aba}$ ， $\text{abbba} \bmod \text{aba} = \text{a}$ 。

现在我们要求出所有 $S_{1,i} \bmod T$ 。

要对每个前缀求解，考虑递推：从 $S_{1,i} \bmod T$ 推出 $S_{1,i+1} \bmod T$ 。（从 $[1, i]$ 的最大匹配段推出 $[1, i + 1]$ 的最大匹配段。）

也就是说，为了完成一次递推，要从 $S \bmod T$ 推出 $(S + c) \bmod T$ ，其中 S 是任意字符串， c 是任意字符。

发现 $(S + c) \bmod T = ((S \bmod T) + c) \bmod T$ 。也就是说， $S + c$ 的最大匹配段的左端点不会超过 S 的最大匹配段。这是因为 $S + c$ 的最大匹配段去掉 c 之后是 S 的一个后缀匹配段，因而不可能超过最大匹配段。

这有什么用呢？我们发现： $S \bmod T$ 一定是 T 的前缀（由定义），所以我们可以把 $((S \bmod T) + c) \bmod T$ 转化为 $(T_{1,k} + c) \bmod T$ ，其中 $k = |S \bmod T|$ 。

换句话说， $S + c$ 的最大匹配段左端点不会超过 S 的最大匹配段，因而可以只考虑这个最大匹配段；而它恰好是 T 的一个前缀。

这样整个过程就和 S 彻底无关了。

现在我们需要对任意的 k, c 求出 $(T_{1,k} + c) \bmod T$ 。也就是在 T 的前缀后面追加一个字符，然后和 T 匹配。（不妨假设 $k \geq 1$ ，否则平凡。）

那么怎么做呢？我们需要另一个性质：

$$S \bmod T = \begin{cases} S & (S \text{ 是 } T \text{ 的前缀}) \\ S_{2,|S|} \bmod T & (\text{otherwise}) \end{cases}$$

即：如果整个字符串不是 T 的某个前缀，那么可以去掉第一个字符之后再算最大匹配段。正确性是显然的。

回到 $(T_{1,k} + c)$ 。如果 $c = T_{k+1}$ 那么就万事大吉了，否则

$$(T_{1,k} + c) \bmod T = (T_{2,k} + c) \bmod T = ((T_{2,k} \bmod T) + c) \bmod T。$$

令 $fail_i = |T_{2,i} \bmod T|$ ，则上式等于

$$(T_{1,fail_i} + c) \bmod T。$$

这两个式子是不是很像？

所以令 $f(k, c) = |(T_{1,k} + c) \bmod T|$ ，那么

$$f(k, c) = \begin{cases} k + 1 & (c = T_{k+1}) \\ 0 & (c \neq T_{k+1}, k = 0) \\ f(fail_k, c) & (c \neq T_{k+1}, k \neq 0) \end{cases}。$$

这样就可以递归计算 f 了。唯一的问题是：我们还没有求出 $fail_i$ 。

该怎么办呢？根据定义： $fail_i = |T_{2,i} \bmod T|$ 。我们发现，对 $T_{2,m}$ 和 T 做 KMP 就可以求解。（KMP 可以对 S, T 求出所有 $S_{1,i}$ 和 T 的最大匹配段。）

（KMP 算法运行过程中需要使用 $fail$ 数组，但这不会导致循环依赖：
 $|T_{2,k} \bmod T| \leq k - 1$ ，所以求解 $fail_{k+1}$ 时只会用到 $fail_1 \sim fail_{k-1}$ 。）

由此我们可以写出如下的代码：

```
int f(int k, char c){
    char goal = (k < m ? t[k+1] : 0);
    if (c == goal) return k+1;
    else if (k == 0) return 0;
    return f(fail[k], c);
}
void getFail(){
    fail[1] = 0;
    for (int i=2; i<=m; ++i) fail[i] = f(fail[i-1], t[i]);
}
```

接下来我们分析一下 KMP 算法的复杂度。

$S_{1,i}$ 的最大匹配段一定在 $S_{1,i+1}$ 最大匹配段的“左边”(左右端点都更小)，因此左端点和右端点的移动距离之和 $\leq 2n$ ；

而 KMP 算法中每一次 `go` 都让左端点或者右端点移动了至少一步，所以查询的时间复杂度是 $O(n)$ 的。

当然我们还需要用 $O(m)$ 的时间求出 *fail* 数组，因此总时间复杂度是 $O(n + m)$ 。

KMP 自动机

发现上述 KMP 算法的核心是 $f(k, c)$ 这个函数，而我们可以在 $O(m|\Sigma|)$ 的时间复杂度之内预处理出 $f(k, c) (0 \leq k \leq m, c \in \Sigma)$ 的取值：对 $k = 0, 1, \dots, m$ 依次计算即可。

这样我们可以连出一个图：节点编号为 $0 \sim m$ ，然后连边 $k \xrightarrow{c} f(k, c)$ 。这就是 KMP 自动机。当我们对一个字符串 S 进行匹配的时候，维护一个节点 u ：

- 初始为根节点 0；
- 然后枚举 $i = 1, \dots, n$ ，令 $u \leftarrow f(u, S_i)$ ，也就是在自动机上沿着对应出边 $u \xrightarrow{S_i} f(u, S_i)$ 走一步。

多串匹配问题

现在模式串不止一条：有 $T_1 \sim T_m$ 这些模式串。如果把 S 放上去直接匹配，那么时间复杂度是 $O(nm)$ 的，一般情况下难以接受。

现在我们发现 KMP 算法中匹配段一定是 T_i 的前缀。有什么东西能维护前缀呢？

没错，就是 Trie。建出 $T_1 \sim T_m$ 的 Trie 树 T ，其中每个节点代表一个字符串。称 $S \in T$ 当且仅当 T 中存在对应 S 的节点。

类似地定义 $S \bmod T$ （其中 T 是 Trie）：最长的 S 的后缀 $S_{i,n}$ 使得 $S_{i,n} \in T$ 。

类似地，尝试求出所有 $S_{1,i} \bmod T$ 。

T 是一个 Trie。令 T_u 表示 Trie 上 u 号节点对应的字符串。

假设 0 号节点为根， $1 \sim m$ 号节点分别代表 $T_1 \sim T_m$ 。

考虑从 $S \bmod T$ 推出 $(S + c) \bmod T$ ，其中 S 是任意字符串， c 是任意字符。

发现在 Trie 上依然有 $(S + c) \bmod T = ((S \bmod T) + c) \bmod T$ 。

所以我们可以把 $((S \bmod T) + c) \bmod T$ 转化为 $(T_x + c) \bmod T$ 。

换句话说， $S + c$ 的最大匹配段左端点不会超过 S 的最大匹配段，因而可以只考虑这个最大匹配段；而它恰好在 T 里面出现过。

这样整个过程就和 S 彻底无关了。

T 是一个 Trie。令 T_u 表示 Trie 上 u 号节点对应的字符串。

假设 0 号节点为根， $1 \sim m$ 号节点分别代表 $T_1 \sim T_m$ 。

现在我们需要对任意的 x, c 求出 $(T_x + c) \bmod T$ 。也就是在 T 中的某个串后面追加一个字符，然后和 T 匹配。（不妨假设 $x \neq 0$ ，否则平凡。）

那么怎么做呢？我们需要另一个性质：

$$S \bmod T = \begin{cases} S & (S \in T) \\ S_{2,|S|} \bmod T & (S \notin T) \end{cases}$$

即：如果整个字符串不是 T 中的串，那么可以去掉第一个字符之后再算最大匹配段。正确性是显然的。

T 是一个 Trie。令 T_u 表示 Trie 上 u 号节点对应的字符串。

假设 0 号节点为根， $1 \sim m$ 号节点分别代表 $T_1 \sim T_m$ 。

回到 $(T_x + c)$ ：如果 $T_x + c \in T$ （即：存在 $x \xrightarrow{c} to_{x,c}$ ）那么就结束了，否则

$$(T_x + c) \bmod T = ((T'_x \bmod T) + c) \bmod T$$

其中 $S' = S_{2,|S|}$ 。

令 $fail_x$ 为 $T'_x \bmod T$ 对应的节点，则上式等于 $(T_{fail_x} + c) \bmod T$ 。

所以令 $f(k, c)$ 表示 $(T_k + c) \bmod T$ 对应的节点，那么

$$f(x, c) = \begin{cases} to_{x,c} & (to_{x,c} \text{ 存在}) \\ 0 & (to_{x,c} \text{ 不存在}, x = 0) \\ f(fail_x, c) & (to_{x,c} \text{ 不存在}, x \neq 0) \end{cases}$$

同样地，我们也只需要求出 $fail_x$ 即可。

T 是一个 Trie。令 T_u 表示 Trie 上 u 号节点对应的字符串。

假设 0 号节点为根， $1 \sim m$ 号节点分别代表 $T_1 \sim T_m$ 。

定义 $S' = S_{2,|S|}$ 。我们需要求出所有 $fail_x$ ，也就是 $T'_x \bmod T$ 。

我们没法把 T 和 T 跑匹配。把 $T_1 \sim T_m$ 依次在 T 上跑匹配也会导致循环依赖。

正确的做法是：对于每条出边 $x \xrightarrow{c} y$ ，在求出 $fail_x$ 过后可以求出 $fail_y = f(fail_x, c)$ 。这是因为 $T'_y = T'_x + c$ 。而为了防止循环依赖，我们按照长度从小到大进行计算，也就是在 BFS 过程中依次求解。

同时为了防止复杂度飞天，求出 $fail_x$ 过后要把所有 $f(x, c)$ 预处理出来。

把 $f(x, c)$ 预处理出来之后我们会得到什么？（三个字）

AC 自动机

很遗憾，它没法帮你自动 AC 题目。

下面提供一份代码。注意这里根节点是 1 号，而不是推导过程中的 0 号。

```
int nV = 1, fail[N], f[N][26], id[N];
void insert(char* s,int x){ // x 为字符串编号
    int n = strlen(s+1), u = 1;
    for (int i=1; i<=n; ++i){
        if (f[u][s[i]-'a']) u = f[u][s[i]-'a'];
        else u = f[u][s[i]-'a'] = ++nV; // 赋值顺序从右到左
    }
    id[u] = x;
}
```

注意：Trie 树和 AC 自动机的 f 共用了同一个数组。想一想为什么这么做是对的？

很遗憾，它没法帮你自动 AC 题目。

```
void build(){
    for (int i=0; i<26; ++i) f[0][i] = 1; // 这里处理了根节点的边界情况
    fail[1] = 0;
    q.push(1);
    while(!q.empty()){
        int u = q.front(); q.pop();
        int lst = fail[u];
        for (int i=0;i<26;++i){
            if (!f[u][i]) f[u][i] = f[lst][i];
            else {fail[f[u][i]] = f[lst][i]; q.push(f[u][i]);}
        }
    }
}
```

这里求出 $fail_x$ 过后没有立即计算 $f(x, c)$ ，不过正确性不会出问题。

例题：P5357 【模板】AC 自动机

给你一个文本串 S 和 m 个模式串 $T_{1 \sim m}$ ，请你分别求出每个模式串 T_i 在 S 中出现的次数。

$$1 \leq m \leq 2 \times 10^5, \sum |T_i| \leq 2 \times 10^5, |S| \leq 2 \times 10^6。$$

例题：P5357 【模板】AC 自动机

给你一个文本串 S 和 m 个模式串 $T_{1\sim m}$ ，请你分别求出每个模式串 T_i 在 S 中出现的次数。

$$1 \leq m \leq 2 \times 10^5, \sum |T_i| \leq 2 \times 10^5, |S| \leq 2 \times 10^6。$$

使用 AC 自动机。建出 Trie 树 T 之后，求出所有 $S_{1,i} \bmod T$ 。但是如何通过这些信息推出每个模式串的出现次数呢？

显然，每个匹配段应该在结尾处累加。如果 $S_{4,6}$ 匹配上了 T_1 ，那么应该在算到 $S_{1,6} \bmod T$ 时给 T_1 的匹配次数 $+1$ 。

而当 $T_1, T_2, T_3 = \text{a}, \text{ba}, \text{aba}$ 时，一个形如 **aba** 的最大匹配段能让这三个字符串的匹配次数都 $+1$ 。

所以需要有一个高效的算法判断哪些位置需要累加。

我们发现：一个字符串 T_k 在 $S_{1,i}$ 处有匹配当且仅当 T_k 是 $S_{1,i} \bmod T$ 的一个后缀。由于 $S_{1,i} \bmod T$ 是 T 中的一个字符串 T_x ，需要设法找出 T_x 在 T 中的所有后缀。

显然 T_x 本身是一个后缀；那么下一个后缀是什么呢？

考虑 T'_x 。当然 T'_x 不一定在 T 当中，需要找出最长的一个在 T 里面的后缀——也就是 $T'_x \bmod T = T_{fail_x}$ 。

也就是说： T_x 的最长的 $\in T$ 的真后缀是 T_{fail_x} 。因此从 x 开始不断跳 $fail$ 就能找出所有 T_x 的后缀。

因此我们连边 $x \rightarrow fail_x (x \neq 0)$ ，在 AC 自动机的顶点上连出一棵树。这被称为 *fail* 树。在匹配时，需要把 $S_{1,i} \bmod T$ 对应节点到根节点的链上匹配次数全部 +1。

当然我们没法直接这么做，因为复杂度会炸。到根节点的路径 +1 可以转化为什么？

没错，单点 +1 然后查询子树和。

算法是：

- 建出 Trie 树 T 和对应的 AC 自动机。
- 在 AC 自动机上进行匹配；把所有 $S_{1,i} \bmod T$ 对应的节点计数器 $+1$ 。
- 然后对计数器数组做 *fail* 树上子树和，得到最终答案。

AC 自动机的空间复杂度、时间复杂度是 $O(|T||\Sigma|)$ ，其中 $|T|$ 为 Trie 大小， $|T| = O(\sum T_i)$ ；对 S 进行匹配的时间复杂度为 $O(n)$ ；最后需要用 $O(|T|)$ 的时间做子树和。

例题：P3121 [USACO15FEB] Censoring G

FJ 把杂志上所有的文章摘抄了下来并把它变成了一个长度不超过 10^5 的字符串 S 。他有一个包含 m 个单词的列表，列表里的 m 个单词记为 $T_1 \cdots T_m$ 。他希望从 S 中删除这些单词。

FJ 每次在 S 中找到最早出现的列表中的单词（最早出现指该单词的开始位置最小），然后从 S 中删除这个单词。他重复这个操作直到 S 中没有列表里的单词为止。注意删除一个单词后可能会导致 S 中出现另一个列表中的单词。

FJ 注意到列表中的单词不会出现一个单词是另一个单词子串的情况，这意味着每个列表中的单词在 S 中出现的开始位置是互不相同的。

请帮助 FJ 完成这些操作并输出最后的 S 。

由于不会出现一个单词是另一个单词子串的情况，匹配段不会出现包含关系；因此左端点更小的段右端点也更小。

先对 $T_1 \sim T_m$ 建 AC 自动机 T 。

所以可以这么做：枚举 $i = 1 \dots n$ ，维护一个最终的字符串 U ，初始为空。

枚举到 S_i 时，令 $U \leftarrow U + S_i$ ，并同步更新 U 在 AC 自动机上的匹配状态。

如果走到的节点包含某个 T_i 作为后缀（预处理：在 *fail* 树上 T_i 的子树打标记），那么 U 尾部必定出现了 T_i （而且它一定是第一个匹配段），把它删掉。

删掉之后 U 变成了 U 的一个前缀，需要找到它在 AC 自动机上的匹配状态。这个前缀在之前的过程中一定出现过，记录一下历史匹配状态即可。

时间复杂度线性。

例题：CF1202E You Are Given Some Strings

给你一个字符串 S 和 m 个字符串 $T_1, T_2, \dots, T_m$ 。所有字符串均由小写英文字母组成。

令 $f(S, T)$ 代表 T 在 S 中出现次数，计算

$$\sum_{i=1}^m \sum_{j=1}^m f(S, T_i + T_j)。$$

$$m, |S|, \sum |T_i| \leq 2 \times 10^5。$$

注：为了记号统一，这里题面中的 S 和 T 与原题的记号相反。

把 $T_i + T_j$ 在分界点处拆开。令 f_i 表示有多少个 T_j 是 $S_{1,i}$ 的后缀， g_i 表示有多少个 T_j 是 $S_{i,n}$ 的前缀，最终答案为 $\sum f_i g_{i+1}$ 。

由对称性，只需考虑 f_i 。显然可以用 AC 自动机求解：建 Trie 树 T 和对应的 AC 自动机。找到 $S_{1,i} \bmod T$ 在 T 上对应的节点 x ，则对 f_i 有贡献的 T_j 是 T_x 的后缀；因而 f_i 等于 x 到根的链（*fail* 树）上终止节点的数量。这个可以对 T 上每个节点预处理。

时间复杂度 $O(|S| + \sum |T_i|)$ 。

例题：CF587F Duff is Mad

给定 n 和 $S_1 \sim S_n$ 。 q 组询问：每次给定 k, l, r ，求

$$\sum_{i=l}^r f(S_k, S_i),$$

其中 f 的定义和上一题相同。

令 $m = \sum |S_i|$ 。 $n, m, q \leq 10^5$ 。

可以把询问离线下来，并转化为两个前缀和 $\sum_{p=1}^i f(S_k, S_p)$ 相减。

第一种思路是先枚举 i 。

对所有字符串建 AC 自动机。之后求 $\sum_{p=1}^i f(S_k, S_p)$ 相当于：把 $S_1 \sim S_i$ 对应的节点 $+1$ ，然后对 S_k 跑一遍匹配（对每个前缀求 $fail$ 树上它到根节点的路径权值和）。

由于可以离线处理询问，依次加入 $S_1 \sim S_n$ ，加入第 i 个字符串时给对应节点 $+1$ ；之后处理该状态对应的询问。

这里涉及单点 $+1$ 、到根的路径求和，可以转化成子树 $+1$ 、单点查询，进而用树状数组维护。

该算法的时间复杂度为 $O((n + q \cdot \max |S|) \log m)$ ；将树状数组替换为 $O(\sqrt{m}) - O(1)$ 分块可以做到 $O(n\sqrt{m} + q \cdot \max |S|)$ 。

然而时间复杂度有平方项，过不了。

我们也有另外一种解决方案：先枚举 k 。

对每个 S_k ，先在 *fail* 树上标记它的所有前缀；之后 $O(m)$ 预处理出所有子树中的标记数量。加入一个字符串 S_i 时，可以直接把答案加上对应节点的权值。

该做法时间复杂度为 $O(nm + q)$ 。

考虑根号分治，把 $|S| > B$ 的字符串拿出来用第二种算法，其余的用第一种算法。

如果使用树状数组，时间复杂度为 $O((n + qB) \log m + \frac{m^2}{B} + q)$ 。取 $B = \frac{m}{\sqrt{q \log m}}$ ，时间复杂度可以达到 $O(n \log m + m \sqrt{q \log m})$ 。

将树状数组替换为 $O(\sqrt{m}) - O(1)$ 分块，调整块长可以做到 $O(n\sqrt{m} + m\sqrt{q})$ 。

例题：P9613 [CERC2019] K==S

给定 $n, m, T_1 \sim T_m$ ，求出长度为 n 的不包含 $T_1 \sim T_m$ 中任何一个作为子串的字符串数量。

$$n \leq 10^9, \sum |T_i| \leq 100。$$

这题涉及一个常用技巧：自动机上 dp。具体地，将节点 x 当作状态 $f_{i,x}$ ，形如 $x \xrightarrow{c} y$ 的边当作（枚举下一个字符 c 之后） $f_{i,x} \rightarrow f_{i+1,y}$ 的转移。

建出 AC 自动机 T 之后，在上面进行 dp。令 $f_{i,x}$ 表示前 i 个字符，当前在自动机上匹配到节点 x 的方案数。

每个 T_i 会禁用 $fail$ 树上子树中所有节点，所以一步转移过后要把它们的 f 清零。

但是本题中 n 非常大，不能直接计算。所以考虑另一个经典技巧：每一步转移的式子都相同，写成矩阵过后可以用矩阵快速幂计算。

时间复杂度 $O((\sum |T_i|)^3 \log n)$ 。

例题：CF1784E Infinite Game

Alice 和 Bob 正在玩一个由若干局组成的无限游戏。每一局由若干轮组成，每一轮有一名玩家获胜。第一个赢得两轮的玩家将赢得该局。因此，每一局的最终比分总是 $2 : 0$ 或 $2 : 1$ ，胜者为 Alice 或 Bob。

我们称一个游戏场景为一个由字符 'a' 和 'b' 组成的有限字符串 s 。考虑一个由 s 无限重复拼接而成的无限字符串： $sss\dots$ 。假设 Alice 和 Bob 按照这个无限字符串从左到右进行每一轮。如果当前字符为 'a'，则 Alice 赢得该轮；如果为 'b'，则 Bob 赢得该轮。当有一名玩家赢得两轮时，该局结束，下一局从下一个字符开始。

一个若干局组成的无限游戏：每一局由若干轮组成，第一个赢得两轮的玩家赢。

一个游戏场景是一个由字符 'a' 和 'b' 组成的有限字符串 s 。考虑一个由 s 无限重复拼接而成的无限字符串： $sss\dots$ 。假设 Alice 和 Bob 按照这个无限字符串从左到右进行每一轮。如果当前字符为 'a'，则 Alice 赢得该轮；如果为 'b'，则 Bob 赢得该轮。当有一名玩家赢得两轮时，该局结束，下一局从下一个字符开始。

定义 a_i 为 Alice 在前 i 局中赢得的局数。定义 r 为比值 $\frac{a_i}{i}$ 在 $i \rightarrow \infty$ 时的极限。如果 $r > \frac{1}{2}$ ，我们称场景 s 对 Alice 有利；如果 $r = \frac{1}{2}$ ，称为平局；如果 $r < \frac{1}{2}$ ，称场景对 Bob 有利。

现在给定一个只包含 'a'、'b' 和 '?' 的字符串 S 。请统计将所有 '?' 替换为 'a' 或 'b' 所有可能的方案中，有多少种方案使得场景对 Alice 有利，有多少种方案为平局，有多少种方案对 Bob 有利。请将这三个数对 998 244 353 取模后输出。

$1 \leq |S| \leq 200$ 。

考虑一个自动机：把目前比分当作状态，胜败当作转移。状态共有 $0:0, 0:1, 1:0, 1:1$ 四种。

这不是典型的自动机 dp，过程中还需要记录 A 和 B 赢了几局。

如果场景 S 固定，那么可以预处理出这四种状态经过一整个 S 之后会转移到哪种状态，途中 A 比 B 多赢了几轮。当 S 无限循环时，一定是从 $0:0$ 开始，每次走一个 S ，然后在环上循环往复。最终的形势优劣只和 A 在一轮循环中比 B 多赢了几局有关。

所以可以进行 dp：让 $f_{i,a,b,c,d,e,f,g,h}$ 代表前 i 步让四种状态分别转移到 a, b, c, d ，比分差增加了 e, f, g, h 。这样做时间复杂度是 $O(n^5)$ 的，但是状态数实际上跑不满，可以卡常通过。

当然还有更优的办法。可以直接枚举最终的环包含哪些节点，这样在 dp 过程中就可以只维护环上 A 比 B 多赢了几局。

令 $f_{i,a,b,c,d,k}$ 代表前 i 步让四种状态分别转移到 a, b, c, d ，比分差是 k 。最后累加答案的时候需要判断是不是真的走到了枚举的那个环上。

时间复杂度 $O(n^2)$ ，但是常数很大 ($4^4 \cdot 2^4$)。

Manacher 算法

令 $r(S)$ 表示 $S_n S_{n-1} \cdots S_1$ 。如果 $S = r(S)$ 则称 S 是回文串。

Manacher 算法能够在 $O(n)$ 的时间复杂度之内找出 S 的所有回文子串。

注：我们只考虑长度为奇数的回文子串。对于偶数的情况，可以在每两个字符之间插入一个相同的占位符，在得到的新串上面算。同时不妨假设 $S_0 \neq S_{n+1}$ 。

考虑对每个位置算出以它为中心的最长回文子串的半径 r_i （定义中心为 i 的回文子串 $s_{l,r}$ 的半径为 $r - i$ ）。我们发现：如果有一个非常长的回文子串，那么可以把它的左边对称到右边，只用算一半的位置。

按照 $1 \sim n$ 的顺序求解，同时维护当前右端点最大的回文串 $[l, r]$ 。

- 如果 $i \leq r$ ，将 r_i 初始化为 $\min(r_{l+r-i}, r - i)$ ；否则初始化为 0。
- 如果 $i + r_i \geq r$ （在实现时不用判断），不断尝试向两侧扩展。
- 最后，更新右端点最大的回文串。

由于每次成功的扩展必定使 r 增大，总时间复杂度是 $O(n)$ 的。

该算法的直接推论是：长度为 n 的字符串只有 $O(n)$ 种不同的回文子串。

以下是参考实现。这里使用了 mx_i 存储回文半径，并且这个回文半径并不是原始串的回文半径，实际使用时需要特殊处理。

```
void manacher(char* S){
    int n = strlen(S+1);
    memset(mx, 0, sizeof(mx));
    static char s[N << 1];
    s[0] = '~'; s[n*2 + 2] = 0; s[1] = '#';
    for (int i=1; i<=n; ++i){
        s[i*2] = S[i];
        s[i*2+1] = '#';
    }
    n = n*2 + 1;
    for (int i=1, r=0, mid=0; i<=n; ++i){
        if (i<=r) mx[i] = min(mx[(mid<<1)-i], r-i+1);
        while(s[i-mx[i]] == s[i+mx[i]]) ++mx[i];
        if (i+mx[i] > r) r = mx[i]+i-1, mid = i;
        --mx[i];
    }
}
```

例题：P4287 [SHOI2011] 双倍回文

记字符串 w 的倒置为 w^R 。例如 $(abcd)^R = dcba$ ， $(abba)^R = abba$ 。

对字符串 x ，如果 x 满足 $x^R = x$ ，则称之为回文。例如 **abba** 是一个回文，而 **abed** 不是。

如果 x 能够写成 ww^Rww^R 形式，则称它是一个“双倍回文”。换句话说，若要 x 是双倍回文，它的长度必须是 4 的倍数，而且 x ， x 的前半部分， x 的后半部分都要是回文。例如 **abbaabba** 是一个双倍回文，而 **abaaba** 不是，因为它的长度不是 4 的倍数。

你的任务是，对于给定的字符串 S ，计算它的最长双倍回文子串的长度。

对于全部数据， $1 \leq n \leq 500000$ 。

注意到长度为 n 的字符串只有 $O(n)$ 种不同的回文子串。

例题：[ABC349G] Palindrome Construction

给定一个长度为 n 的非负整数序列 $A = (A_1, A_2, \dots, A_n)$ 。请判断是否存在一个满足下述条件的长度为 n 的正整数序列 $S = (S_1, S_2, \dots, S_n)$ ，如果存在，请输出所有满足条件的序列中字典序最小的一个。

对于每个 $i = 1, 2, \dots, n$ ，都满足以下两点：

- 序列 $(S_{i-A_i}, S_{i-A_i+1}, \dots, S_{i+A_i})$ 是回文的。
- 如果 $2 \leq i - A_i$ 且 $i + A_i \leq n - 1$ ，则序列 $(S_{i-A_i-1}, S_{i-A_i}, \dots, S_{i+A_i+1})$ 不是回文的。

$$1 \leq n \leq 2 \times 10^5。$$

一种朴素的想法是：把回文的限制转化为形如 $S_u = S_v$ 和 $S_u \neq S_v$ 两种。前一类限制用并查集缩点，然后按位贪心。

具体地：枚举 $i = 1 \dots n$ ，如果 S_i 还未被确定，枚举连向这个连通块的所有形如 $S_u \neq S_v$ 的限制，选择这些限制没有排除的最小正整数。第二类限制至多只有 n 个，所以这部分的时间复杂度正确。

问题在于第一类限制太多，需要优化。

考虑模拟 Manacher 算法的过程：

- 枚举 $i = 1 \dots n$ ，动态维护当前右端点最大的回文子串 $[l, r]$ ；
- Manacher 算法首先会根据 r_{l+r-i} 计算 r_i 的初始值，如果这个值 $> A_i$ ，直接返回无解；
- 否则，如果需要进行扩张，每次扩张时 $r_i \leftarrow r_i + 1$ 然后合并 $i - r_i, i + r_i$ 。扩张直到 $r_i = A_i$ 。
- 最后限制 $S_{i+A_i+1} \neq S_{i-A_i-1}$ 。

该算法为什么正确呢？

考虑归纳证明：处理完 $1 \sim k$ 以后，对于所有的 $1 \leq p \leq k, 1 \leq q \leq A_p$ ， $p - q$ 和 $p + q$ 在并查集中已被合并。

如果对 $1 \sim k$ 成立，那么处理 $p = k + 1$ 时，先通过 Manacher 算法算出了初始的 r_p 。对 $q \leq r_p$ ，有 $A_{p+q} \sim A_{(l+r)-(p+q)} \sim A_{(l+r)-p+q} \sim A_{p-q}$ ，所以 $p + q$ 和 $p - q$ 已经被合并。

对于 $r_p < q \leq A_p$ ，在 Manacher 的扩张过程中会把 $p + q$ 和 $p - q$ 合并。

所以最后并查集的状态是正确的。总时间复杂度 $O(n\alpha(n))$ 。

后缀数组

一个字符串的所有前缀是好维护的，但是子串情况比较复杂，难以维护。

直接处理子串的时间复杂度是平方级别的。但是子串一定是后缀的前缀，所以只要能高效地维护后缀，那么就可以处理子串问题。

一种方法是后缀数组，它维护了所有后缀按照字典序排序后的结果。

如何进行后缀排序呢？直接比较显然行不通，考虑倍增：先比较每个后缀的第一个字符，然后比较前两个字符，然后前四个，以此类推。为什么这么做行得通？

这是因为比较 $S_{i,n}$ 和 $S_{j,n}$ 的前 2^k 个字符时，可以先比较它们的前 2^{k-1} 个字符，如果相等再对 $S_{i+2^{k-1},n}$ 和 $S_{j+2^{k-1},n}$ 进行比较，也只需要比前 2^{k-1} 个字符。

所以我们可以递推出只考虑前 2^k 个字符的情况下，所有后缀的排名。每次递推只需要对上一轮排名组成的二元组 $(rk_i, rk_{i+2^{k-1}})$ 进行排序，时间复杂度可以做到 $O(n \log n + |\Sigma|)$ 。

下面是一份参考实现。最后 sa_i 代表第 i 小的后缀编号, rk_i 代表 $S_{i,n}$ 的排名。

```
void buildSA(char* s){
    int n=strlen(s+1),m=128,p;
    static int id[N],cnt[N],oldrk[N<<1];
    memset(cnt,0,sizeof(cnt));
    memset(oldrk,0,sizeof(oldrk));
    memset(id,0,sizeof(id));
    for (int i=1;i<=n;++i) ++cnt[rk[i]=s[i]];
    for (int i=1;i<=m;++i) cnt[i]+=cnt[i-1];
    for (int i=n;i;--i) sa[cnt[rk[i]]--]=i;
    for (int w=1;;w<=1,m=p){
        int now=0;
        for (int i=n-w+1;i<=n;++i) id[++now]=i;
        for (int i=1;i<=n;++i) if (sa[i]>w) id[++now]=sa[i]-w;
        memset(cnt,0,sizeof(cnt));
        for (int i=1;i<=n;++i) ++cnt[rk[i]];
        for (int i=1;i<=m;++i) cnt[i]+=cnt[i-1];
        for (int i=n;i;--i) sa[cnt[rk[id[i]]]--]=id[i];
        p=0;
        memcpy(oldrk,rk,sizeof(rk));
        for (int i=1;i<=n;++i){
            if (oldrk[sa[i]]==oldrk[sa[i-1]]&&oldrk[sa[i]+w]==oldrk[sa[i-1]+w]) rk[sa[i]]=p;
            else rk[sa[i]]=++p;
        }
        if (p==n) break;
    }
}
```

后缀数组有什么用途呢？

如果一个字符串中有两个相同的子串 $S_{l,r} = S_{x,y}$ ，那它们可能分散在任意两个地方。但是如果进行后缀排序，那么 $S_{l,n}$ 和 $S_{x,n}$ 就会排在一起——因为它们前几个字符相同，字典序“差不多”。

所以启发我们定义 h_i 表示 $S_{sa_i,n}$ 和 $S_{sa_{i-1},n}$ 的最长公共前缀的长度。由字典序的性质， $S_{sa_l,n}$ 和 $S_{sa_r,n}$ ($l < r$) 的最长公共前缀长度就是

$$\min_{i=l+1}^r h_i。$$

用 ST 表查询区间最小值，预处理时间复杂度 $O(n \log n)$ 和计算后缀数组的复杂度相同；单次查询可以做到 $O(1)$ 。

我们枚举 $i = 1 \dots n$ ，这是因为 $h_{rk_{i+1}} \geq h_{rk_i} - 1$ 。

证明：考虑 i 和（后缀数组中）排在它前面的后缀 j 。 h_{rk_i} 表示 $S_{i,n}$ 和 $S_{j,n}$ 的 LCP（最长公共前缀）长度。

所以 $S_{j+1,n}$ 和 $S_{i+1,n}$ 的 LCP 长度为 $h_{rk_i} - 1$ 。而它等于

$$\min_{p=rk_{j+1}+1}^{rk_{i+1}} h_i \leq h_{rk_{i+1}} \circ$$

```
for (int i=1,k=0;i<=n;++i){
    if (rk[i]==1) continue;
    if (k) --k;
    while(s[i+k]==s[sa[rk[i]-1]+k]) ++k;
    height[rk[i]]=k;
}
```

后缀数组是所有后缀按照字典序排序后的结果。 sa_i 代表第 i 小的后缀编号, rk_i 代表 $S_{i,n}$ 的排名。 h_i 表示 $S_{sa_i,n}$ 和 $S_{sa_{i-1},n}$ 的 LCP 的长度。

如何求出 S 的本质不同子串数量?

按字典序依次考虑每个后缀的贡献 (子串是后缀的前缀): 前 h_i 个前缀和上一个后缀的前缀重复了, 因此答案要累加上 $(n - sa_i + 1) - h_i$ 。全部加起来之后得到 $n(n - 1)/2 - \sum h_i$ 。

为什么只考虑相邻两个后缀的重复段就可以保证不重不漏?

后缀数组是所有后缀按照字典序排序后的结果。 sa_i 代表第 i 小的后缀编号, rk_i 代表 $S_{i,n}$ 的排名。 h_i 表示 $S_{sa_i,n}$ 和 $S_{sa_{i-1},n}$ 的 LCP 的长度。

如何求出 T 在 S 中所有的出现位置?

考虑枚举 $i = 1 \dots m$, 同时动态维护 T 可能的出现位置 (匹配段开头)。由于后缀数组中后缀按照字典序排序, 这些可能的出现位置是后缀数组上一个区间 $[l, r]$ 。

在处理 T_i 时, 在 $[l, r]$ 上二分出可以匹配第 i 位的后缀的区间 $[L, R]$, 令 $l, r \leftarrow L, R$ 。

最后区间中的后缀就是可能的出现位置。

这个区间有什么性质呢? $h_{l+1} \sim h_r$ 都 $\geq m$; 同时 $h_l, h_{r+1} < m$ 。

我们可以对 h_i 建笛卡尔树：具体地，每次把最小的 h_i 取出来，用它把当前区间 $[l, r]$ 分裂成 $[l, i - 1]$ 和 $[i, r]$ 递归构造。如果最小值有多个，则一次性分裂出多个区间。（代码中不特判一般也没问题。）

可以发现笛卡尔树上每个区间代表一类“相似”的后缀。分裂过后，得到的两个区间中的后缀彼此之间更“相似”。（这里相似程度指的是拥有多长的 LCP。）

一些拓展：这个笛卡尔树正是后缀树。容易发现，它也是把所有后缀建 Trie 之后压缩（将只有一条出边的非终止节点 $x \xrightarrow{c} y$ 合并到它指向的节点 y ）得到的结果。

事实上， S 每个子串在 S 中的出现位置必定是笛卡尔树上某个区间（对应后缀树上一个节点）。因此子串的出现位置只有至多 $2n - 1$ 类。

将子串按照出现位置的右端点集合（对反串建后缀数组即可求出）划分，末尾追加字符作为转移可以得到**后缀自动机**，不过它超出了今天的讨论范围。后缀数组和后缀自动机的用途相似，许多时候可以互相替代。

例题：P3181 [HAOI2016] 找相同字符

给定两个字符串 S, T ，求出在两个字符串中各取出一个子串使得这两个子串相同的方案数。两个方案不同当且仅当这两个子串中至少有一个位置不同。

$1 \leq n, m \leq 2 \times 10^5$ ，字符串中只有小写字母。

对于多个串，一种处理方法是建 Trie；但是要维护子串信息，所以 Trie 不好用，更好的办法是用分隔符连接，对 $S + \# + T$ 建后缀数组。

在一个字符串中取出两个位置不同的相同子串的方案数是好算的。转化到后缀数组的 h_i 上类似矩形计数，可以用单调栈求解。

然后就简单了：对 S 算一次，对 T 算一次，然后对 $S + \# + T$ 算一次，最后容斥一下。

例题：P9482 [NOI2023] 字符串

给定字符串 S ，有 q 次询问，每次给定 i, r ，求出有多少个 l 满足 $1 \leq l \leq r$ 且 $S_{i,i+l-1} < r(S_{i+l,i+2l-1})$ 。

$1 \leq T \leq 5$ ， $1 \leq n \leq 10^5$ ， $1 \leq q \leq 10^5$ ， $1 \leq i + 2r - 1 \leq n$ ，字符串 S 仅包含小写字母。

把询问离线下来。子串的字典序不好处理，什么的字典序好处理？

后缀的字典序好处理。所以可以先对 $S + \# + r(S)$ 建后缀数组，这样就可以比较 $S_{i,n}$ 和 $r(S_{1,i+2l-1})$ 。

具体地，倒序枚举后缀数组中的元素，遇到 $r(S)$ 的后缀时在对应位置 $+1$ ，遇到 $S_{i,n}$ 时关于它的询问进行区间查询。可以用树状数组分奇偶性维护。

当然我们还需要处理 $S_{i,i+l-1} = r(S_{i+l,i+2l-1})$ 且 $S_{i,n} < r(S_{1,i+2l-1})$ 的情况。

先用 Manacher 找出所有偶回文子串。假设中心为 $i-1, i$ 的最长回文串长度的一半为 R_i 。

如果 $S_{i,i+l-1} = r(S_{i+l,i+2l-1})$ 且 $S_{i,n} < r(S_{1,i+2l-1})$ ，那么必定有 $R_{i+l} \geq l$ 且 $S_{i+l,n} < r(S_{1,i+l-1})$ 。

所以对每个中心单独处理：处理到 $(i-1, i, R_i)$ 时如果 $S_{i,n} < r(S_{1,i-1})$ 则对 $(i-1, 1) \sim (i-R_i, R_i)$ 这条 45° 直线 -1 ，然后进行 $(i, 1) \sim (i, r)$ 的横线查询。

斜线修改横线查询考虑做线性变换，转化为竖线修改横线查询，然后上树状数组。

这样就完成了求解，总时间复杂度 $O(n \log n)$ 。

The End